Predicting GEMM soft-error resilience from the input distribution with SOGA

Proof of concept, and a proposed NVBitFI validation on GPU

Emilio Incerto, note for Lishan Yang

October 26, 2026

In one sentence. We write the GEMM kernel *and* the NVBitFI fault injector as a single SOGA probabilistic program. From the input distribution alone, SOGA computes the probability that a fault is masked: no injections, no statistical noise, plus which register and which bit are responsible. A GPU campaign is then needed only to confirm the prediction.

1. The question

A fault-injection (FI) campaign estimates resilience **for one input**, with about ±3 % at 95 % confidence after 1000 injections. Typhoon proposes the Frobenius norm of the input as a predictor of resilience. We ask two questions. Can resilience be **computed** from the input *distribution*, without injecting? And does the norm determine it?

2. What we model

Table 1: The modelled setup.

Kernel	PolyBench-GPU GEMM (`CUDA/GEMM/gemm.cu`), float32, $512 \times 512 \times 512$, $\alpha = 32412$, $\beta = 2123$. One thread per output cell: `acc += (alpha*a[i][k]) * b[k][j]` (one FMUL and one FFMA per iteration).
Inputs	A is drawn from one of four distributions (Table 2). B and the initial C are fixed $U[0, 1]$ matrices.
Fault model	The NVBitFI default campaign: `G_GP` + `FLIP_SINGLE_BIT`. A uniformly random dynamic instruction is chosen, and a uniformly random bit (0 to 31) of its destination register is flipped.
Outcome	The `gemm.cu` check. SDC if any output element differs from the CPU reference by more than 0.05 %; masked otherwise; DUE on a crash or hang. A NaN output *passes* the check (`NaN > threshold` is false).
What SOGA covers	The **arithmetic** destination registers: loads of a and b, FMUL αa, and the FFMA accumulator. Index, address and loop-counter registers are *not* modelled; the GPU campaign measures them.

Table 2: Input distributions. The first three have the **same** Frobenius norm.

A distribution	definition	$\|A\|_F$
U[0,1]	uniform on $[0, 1]$ (SUGAR's GEMM input)	296
N(0,1/3)	Gaussian, mean 0, variance 1/3	296
Sparse10%	0 with probability 0.9, otherwise uniform on $[0, \sqrt{10}]$	296
U[0,10]	uniform on $[0, 10]$ (SUGAR's MVT range)	2959

3. One SOGA program = one NVBitFI campaign

The program follows the four steps of a campaign. This is a simplified excerpt of the generated program, with one branch shown (the full program is in `results/unified_2026-10-26/`):

```
site = gm([1/4, 1/4, 1/4, 1/132, ...], [0, 1, 2, 3, ...], [0, ...]);  // 1. which instruction is hit
if site == 2 {                                                         //    here: FMUL t = alpha*a
  a = gm([...]);  b = gm([...]);                                       // 2. input distributions
  t = 32412 * a;  T = t * b;  R = gauss(...);  C = T + R;              //    golden output cell C
  if t >= 8192 {                    // 3. t >= 2^13 (binade [2^13,2^14) = this minus a t >= 2^14 branch)
    kind = gm([23/32, 9/32], [0, 1], [0, 0]);                          //    mantissa | sign+exponent
    if kind == 0 {
      f = gm([1/23, ...], [2^(13-23+0), ..., 2^(13-23+22)], [0, ...]);  //  mantissa bit b
      err = f * b;                                                     //    error reaching C
      if 0.0005*C - err >= 0 { masked = 1; } else { masked = 0; } end if;   // 4. gemm.cu check
    } else { ... } end if;
  } ...
} else { ... other instructions ... } end if;
```

1. **Which instruction.** `site` is a discrete random variable, weighted like the per-thread instruction counts. It covers loads of a and b, the FMUL, and the FFMA accumulator at 33 loop positions.
2. **Kernel.** The input distributions are Gaussian mixtures (SOGA's EM preprocessor). The program computes the golden output cell C. The other 511 terms of the dot product are one moment-matched Gaussian R.
3. **Which bit.** The program branches on the binade $[2^e, 2^{e+1})$ of the hit register, so its IEEE-754 exponent is known. It then picks one of the 32 bits with probability 1/32 each:
   - a mantissa bit gives an additive error 2^{e-23+b};
   - the sign bit or an exponent bit gives a multiplicative error;
   - an exponent that reaches 255 gives NaN (masked).
4. **Check.** The `gemm.cu` threshold.

SOGA returns $E[\texttt{masked}] = P(\text{masked})$. Grouping SOGA's output components by `site` and `kind` gives the rate **per register and per bit class from the same solve**.

Table 3: The single program reproduces the separate per-register solves to within $5 \cdot 10^{-5}$ in total ($1.4 \cdot 10^{-4}$ per register). Single-core solve at $\varepsilon = 5 \cdot 10^{-4}$, $n = 512$.

A	P(masked), single program	same, 36 separate programs	single program: time, components, peak memory
U[0,1]	0.674511	0.674514	198 s, 328k, 2.0 GB
U[0,10]	0.671400	0.671403	197 s, 326k, 1.7 GB
Sparse10%	0.829466	0.829473	915 s, 1.21M, 3.4 GB
N(0,1/3)	0.554227	0.554274	4053 s, 5.26M, 4.7 GB

Validation oracle. A CPU emulator of the NVBitFI campaign. It runs the same kernel, replays the faulty thread with exact float32 FMUL/FFMA semantics, applies the same check, uses one input instance per distribution, and performs 30,000 injections.

4. Results

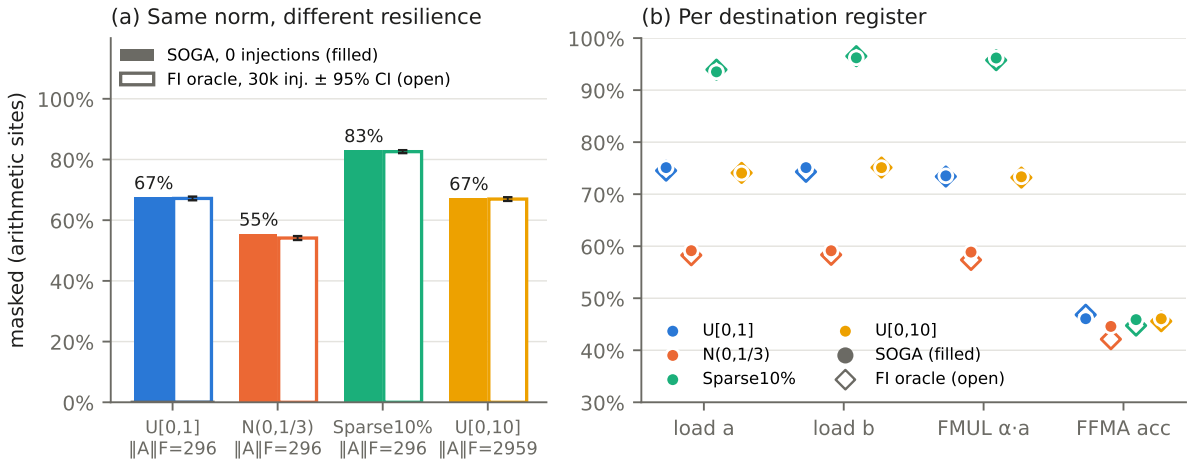


Figure 1: **(a)** Masked rate on arithmetic registers. Filled: SOGA, from the input distribution only. Open: FI oracle on one input instance, with 95 % CI. The first three distributions have the same Frobenius norm. **(b)** The same rate per destination register.

Table 4: Masked rate, SOGA / FI oracle, at the `gemm.cu` threshold. Oracle 95 % CI: about ±1.2 pp per register and ±0.6 pp for the aggregate.

A	load a	load b	FMUL αa	FFMA acc	**all arithmetic**
U[0,1]	.751 / .745	.751 / .743	.735 / .734	.461 / .468	**.675 / .672**
N(0,1/3)	.591 / .583	.591 / .583	.589 / .574	.446 / .421	**.554 / .541**
Sparse10%	.935 / .939	.962 / .965	.962 / .958	.459 / .447	**.829 / .826**
U[0,10]	.741 / .741	.751 / .751	.733 / .732	.461 / .456	**.671 / .670**

What we learn.

1. **The norm does not determine resilience.** At the same $\|A\|_F$ the masked rate ranges from 55 % to 83 %. A 10× larger norm changes nothing (67.5 % vs 67.1 %). With a relative check, resilience is scale-invariant: it depends on the **shape** of the distribution.
2. **Mechanism.** The FFMA accumulator is the weak point, about 45 % masked for *every* distribution, because it carries a growing share of the output. The distribution acts through the registers that hold a single term (loads, FMUL):
   - a zero entry (Sparse) almost always masks its flips;
   - a zero-mean input makes the output small relative to its terms through sign cancellation, so faults are masked less often.
3. **Precision per solve.** SOGA's deviation from the oracle is at most 1.4 pp (at most 0.3 pp for three of the four distributions). That is the precision of roughly 5,000 to more than 30,000 injections per input, obtained from one deterministic solve.

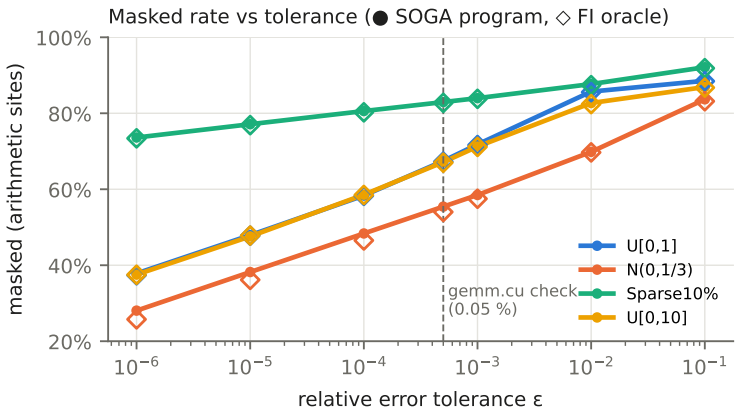


Figure 2: Masked rate vs tolerance ε. ● lines: the SOGA program, solved at each of the seven ε values. ◇: FI oracle. The deviation is at most 0.5 pp for U[0,1], Sparse10% and U[0,10], and at most 2.3 pp for N(0,1/3), largest at small ε. On the GPU, one campaign that logs each run's maximum relative error measures this whole curve.

5. What we ask you to run on the GPU

Minimal changes to `gemm.cu`.

1. `init()`: draw A from the chosen distribution (command-line name and seed). Draw B and C from $U[0, 1]$ with a fixed seed that is the same for all campaigns.
2. Remove the run-time prints from stdout, so the golden stdout is deterministic.
3. After `compareResults`, write max $|C_{gpu} - C_{cpu}|/|C_{cpu}|$, and the number of NaN/Inf elements, to a side file that `sdc_check.sh` does **not** compare. This gives the tolerance curve.

Table 5: Runs per input distribution (four distributions).

	NVBitFI run	injections	purpose
required	profiler on `gemm_kernel`	1	real instruction mix per opcode
required	`G_GP` + `FLIP_SINGLE_BIT`	1000	your standard campaign
recommended	`G_FP32` + `FLIP_SINGLE_BIT`	1000	FMUL / FFMA registers (±3 pp)
recommended	`G_LD` + `FLIP_SINGLE_BIT`	1000	load registers (±3 pp)

What to send back, per injection: the opcode, the bit, the outcome (all in the NVBitFI logs), and the side-file maximum relative error.

Analysis. Take the injections whose opcode is FFMA, FMUL or LDG; their masked fraction is the quantity SOGA predicts. The other `G_GP` injections (integer, address, loop control) are the structural part, which your campaign measures and SOGA does not model.

Table 6: Pre-registered SOGA predictions (masked, $\varepsilon = 5 \cdot 10^{-4}$), made before any GPU run.

A	arithmetic (`G_GP`, filtered)	FFMA	FMUL	loads
U[0,1]	67.5 %	46 %	74 %	75 %
N(0,1/3)	55.4 %	45 %	59 %	59 %
Sparse10%	82.9 %	46 %	96 %	94–96 %
U[0,10]	67.1 %	46 %	73 %	74–75 %

The prediction holds if:

1. each value above lies inside the 95 % CI of the GPU measurement;
2. the ordering Sparse10% > U[0,1] $\approx$ U[0,10] > N(0,1/3) is reproduced;
3. U[0,1] and U[0,10] are equal within the CI.

Statistics. About 760 of 1000 `G_GP` injections land on arithmetic registers (assumed mix), which gives about ±3.5 pp. The predicted gaps of 12 to 28 pp are therefore clearly detectable. The *equality* in criterion 3 is tested only to about ±5 pp; ±1 pp needs about 17,000 arithmetic injections per distribution.

6. Limitations

- **Instruction mix.** The weights of `site` come from a per-thread instruction mix that we assumed from the CUDA source. Your profiler output replaces it; the per-register predictions do not depend on it.
- **Value registers only.** Faults in index, address and loop-counter registers, and crashes, are not modelled.
- **What the validation shows.** The oracle is our CPU emulator, which shares the fault model. It validates SOGA's approximations, not hardware realism; the GPU campaign adds that.
- **Approximations in the SOGA program.**
  - Input distributions are 8-component Gaussian mixtures.
  - The rest of the dot product is one moment-matched Gaussian.
  - Only the magnitude of a mantissa error is used, not its sign.
  - Binades with probability below 10^{-5} are dropped.
  - FFMA is evaluated at 33 of the 512 loop positions.
  - SOGA conditions on half-spaces, so "the register lies in a binade" is written as the difference of two one-sided conditions.
- **The Gaussian input needed a fix.** For N(0,1/3) a single wide Gaussian gave +5.6 pp. Splitting it into 24 narrow components (same mean and variance) reduced the error to +1.4 pp. FFMA under N(0,1/3) still deviates by +2.5 pp.
- **No speed-up claim.** Solve time is not optimized: on a single core, the single program takes 3 to 15 minutes per distribution for the uniform and sparse inputs, and 68 minutes for the Gaussian one, at one ε. We have not measured the wall time of a GPU campaign, so the claim is precision per solve and explanation, not speed.

Reproducibility: `experiments/lishan_gemm_nvbitfi_poc_2026-10-26/` in the SOGA repository. The single programs are in `results/unified_2026-10-26/`, the separate solves in `results/2026-10-26_16-06-40`, `_16-15-23` and `_17-07-11`, and the figures come from `make_figures.py`.